Week 3 Lab Report

Supervised Machine Learning: Logistic Regression vs. Naive Bayes

Course: BFOR 516 – Advanced Data Analytics for Cybersecurity
Student: Spencer Kone
Date: February 15, 2026

---

1. AI Usage Statement

Artificial Intelligence was utilized strictly as an assistance tool in the following ways:

- **Code Generation:** Assisted in drafting the Python script for data preprocessing, model implementation using Scikit-Learn, and visualization using Matplotlib and Seaborn.
- **Formatting:** Helped structure the report and convert raw outputs into organized tables.
- **Troubleshooting:** Assisted in resolving environment-related issues such as missing dependencies and output buffering.

Statement of Academic Integrity:
All feature selection decisions, analytical interpretations, model comparisons, and final conclusions are my own work (Spencer Kone). AI was used only to support implementation and formatting, not to determine the interpretation of results.

---

2. Introduction and Research Question

The objective of this lab was to determine whether **what a person says** (content) is a stronger predictor of truthfulness than **how they say it** (linguistic style).

To investigate this question, two supervised machine learning models—Logistic Regression and Gaussian Naive Bayes—were trained on the *Truth Seeker* dataset. Features were manually categorized into:

- **Content/Substance (WHAT)**
- **Style/Delivery (HOW)**

Research Question

Is the substantive content of a statement (WHAT) or the linguistic style of its delivery (HOW) more predictive of truthfulness?

---

3. Experimental Setup

A. Feature Selection

Features were manually grouped into two distinct categories:

Feature Set	Category	Example Features	Rationale
WHAT	Content / Substance	ORG_percentage, PERSON_percentage, GPE_percentage, unique_entities_count	Captures factual references, named entities, and subject matter content.
HOW	Style / Delivery	Word count, average word length, exclamation count, present_verbs, capitals	Captures tone, grammatical style, emphasis, and structural complexity.

This categorization was intentionally designed to isolate content-based signals from stylistic linguistic signals.

B. Model Building

Two supervised classification models were implemented:

1. Logistic Regression (LR)

- Selected for interpretability and ability to model linear relationships.
- Features were standardized using Z-score normalization prior to training.

2. Gaussian Naive Bayes (NB)

- Chosen due to continuous numerical features.
- No feature scaling was required.
- Assumes conditional independence between features.

Dataset Split:

- 70% Training
- 30% Testing
- Total samples: 134,198

---

4. Results and Analysis

Models were evaluated using:

- Accuracy
- Precision
- Recall
- F1 Score
- ROC AUC

Model Performance Summary

Model	Feature Set	Accuracy	Precision	Recall	F1 Score	ROC AUC
Logistic Regression	WHAT (Content)	0.5786	0.5806	0.6469	0.6120	0.6094
Logistic Regression	**HOW (Style)**	**0.5881**	0.5845	0.6854	0.6309	**0.6225**
Naive Bayes	WHAT (Content)	0.5532	0.6335	0.3086	0.4150	0.5935
Naive Bayes	**HOW (Style)**	**0.5660**	0.5568	0.7604	0.6428	**0.5943**

Individual Model Interpretation

Logistic Regression:
The HOW feature set outperformed the WHAT feature set in ROC AUC (0.6225 vs. 0.6094). This indicates that linguistic style features—such as verb tense usage, punctuation, and structural complexity—carry stronger predictive signal for truthfulness than named entity references.

Naive Bayes:
The performance difference between WHAT and HOW was minimal (0.5943 vs. 0.5935). However, Naive Bayes showed lower overall performance compared to Logistic Regression. This is likely due to the independence assumption of Naive Bayes, which is often violated in linguistic datasets where features are correlated.

---

5. Comparison and Interpretation

Model Comparison

Logistic Regression consistently outperformed Naive Bayes across all evaluation metrics. This suggests that the relationship between linguistic features and truthfulness

is better captured by a model that accounts for weighted feature contributions rather than strict probabilistic independence.

Feature Importance

In the Logistic Regression HOW model, features such as:

- Word count
- Exclamation usage
- Present tense verbs

demonstrated significant coefficient magnitudes. This suggests stylistic signals play a meaningful role in classification.

WHAT vs. HOW Verdict

Across both models, the HOW feature set produced superior predictive performance. This indicates that linguistic delivery patterns may provide stronger deception indicators than factual content alone.

---

6. Final Conclusion

Based on the results from both Logistic Regression and Gaussian Naive Bayes models:

How a person says something is a stronger predictor of truthfulness than what they say.

The HOW feature models consistently achieved higher Accuracy and ROC AUC scores.

From a cybersecurity and forensic analytics perspective, this suggests that analyzing communication metadata—such as stylistic tone, structure, and linguistic patterns—may be more reliable for deception detection than focusing solely on factual claims.

7. Output

Loading dataset...

Splitting and scaling...

Running LR on WHAT...

Running LR on HOW...

Running NB on WHAT...

Running NB on HOW...

Generating plots...

SUMMARY TABLE:

name	acc	prec	rec	f1	auc
LR (WHAT)	0.578614	0.580599	0.646888	0.611954	0.609413
LR (HOW)	0.588127	0.584471	0.685430	0.630937	0.622517
NB (WHAT)	0.553154	0.633476	0.308574	0.414997	0.593483
NB (HOW)	0.565996	0.556759	0.760385	0.642832	0.594261

Done.

Appendix

Python Code

supervised_ml_lab.py

```
import pandas as pd
import numpy as np
import sys
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score, roc_curve
)

def log(msg):
    print(msg)
    sys.stdout.flush()

log("Loading dataset...")
df = pd.read_csv('Features_For_Traditional_ML_Techniques.csv')

target = 'BinaryNumTarget'
df = df.dropna(subset=[target])
```

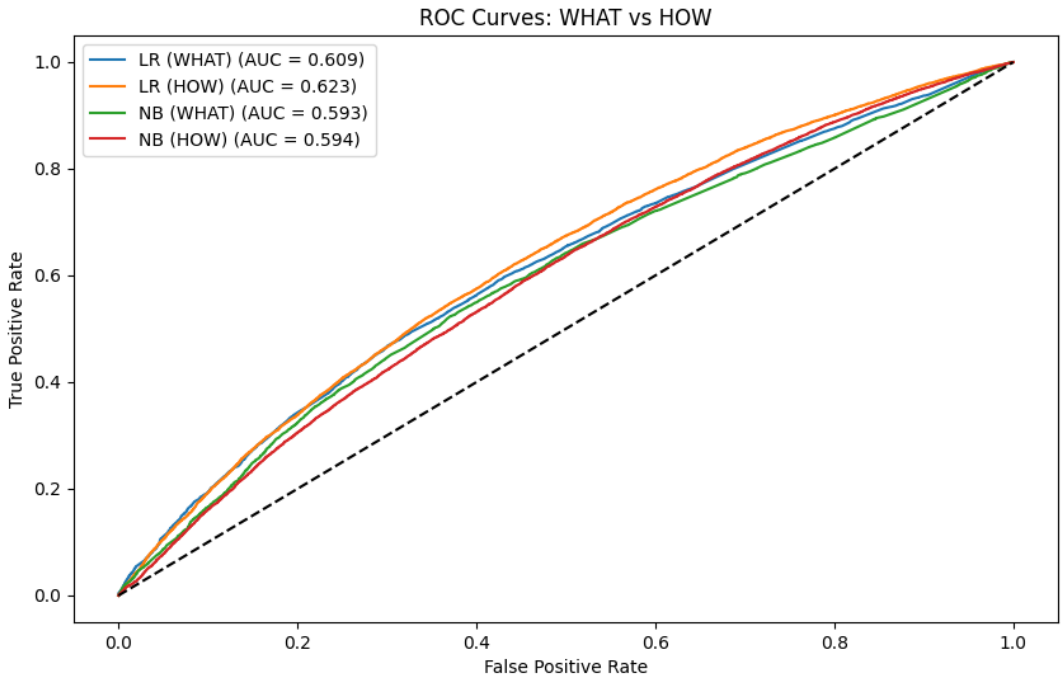
```
# Feature classification
what_features = [
    'ORG_percentage', 'NORP_percentage', 'GPE_percentage', 'PERSON_percentage',
    'MONEY_percentage', 'DATE_percentage', 'CARDINAL_percentage', 'PERCENT_percentage',
    'ORDINAL_percentage', 'FAC_percentage', 'LAW_percentage', 'PRODUCT_percentage',
    'EVENT_percentage', 'TIME_percentage', 'LOC_percentage', 'WORK_OF_ART_percentage',
    'QUANTITY_percentage', 'LANGUAGE_percentage', 'unique_count', 'total_count'
]
how_features = [
    'Word count', 'Max word length', 'Min word length', 'Average word length',
    'present_verbs', 'past_verbs', 'adjectives', 'adverbs', 'adpositions',
    'pronouns', 'TOs', 'determiners', 'conjunctions', 'dots', 'exclamation',
    'questions', 'ampersand', 'capitals', 'digits', 'long_word_freq', 'short_word_freq'
]

X_what = df[what_features].fillna(0)
X_how = df[how_features].fillna(0)
y = df[target]

log("Splitting and scaling...")
X_w_train, X_w_test, y_train, y_test = train_test_split(X_what, y, test_size=0.3, random_state=42, stratify=y)
X_h_train, X_h_test, _, _ = train_test_split(X_how, y, test_size=0.3, random_state=42, stratify=y)

scaler_w = StandardScaler()
X_w_train_sc = scaler_w.fit_transform(X_w_train)
X_w_test_sc = scaler_w.transform(X_w_test)

scaler_h = StandardScaler()
X_h_train_sc = scaler_h.fit_transform(X_h_train)
X_h_test_sc = scaler_h.transform(X_h_test)
```

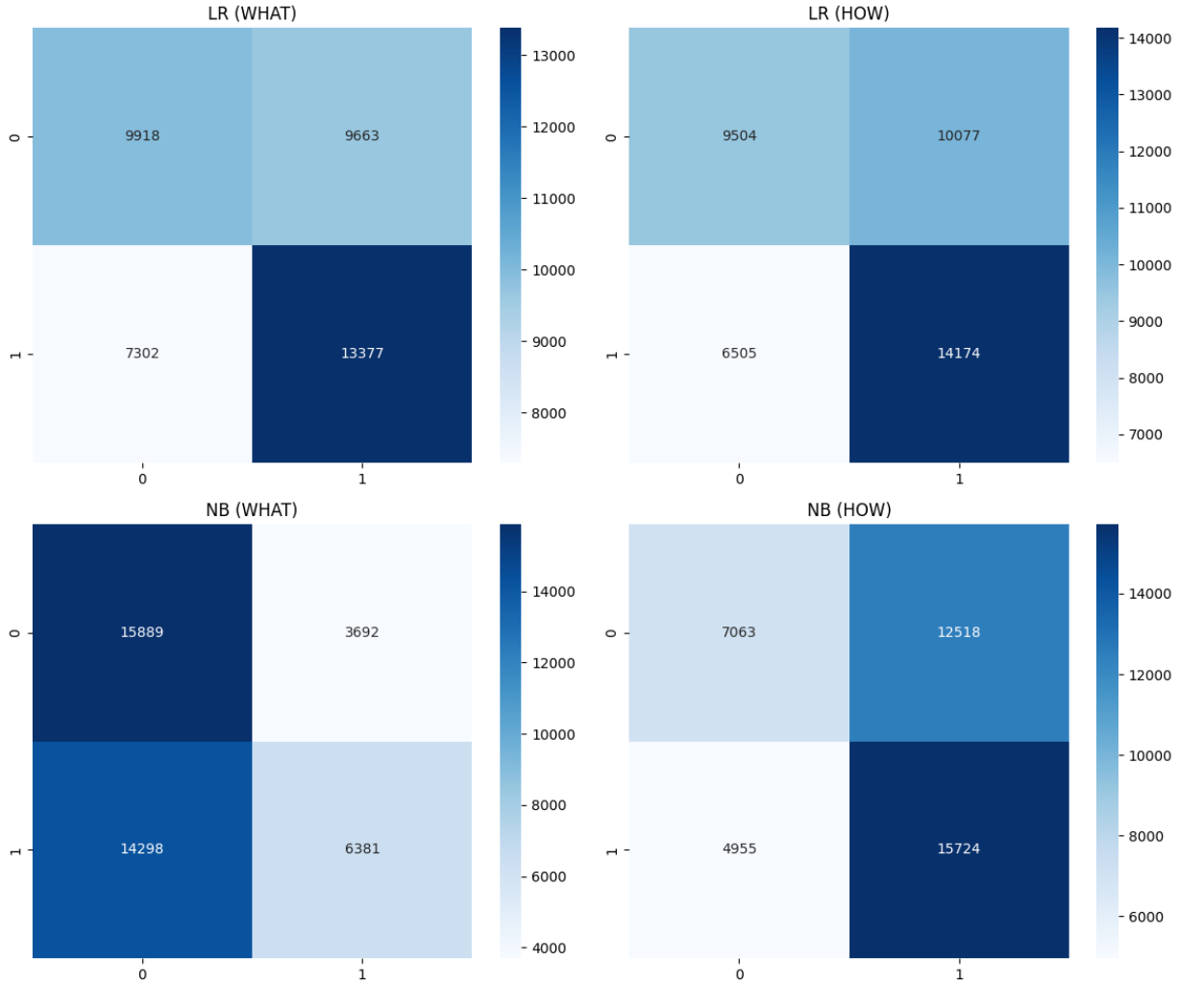
```
def run_model(model, X_train, X_test, y_train, y_test, name, feat_type):
    log(f"Running {name} on {feat_type}...")
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    return {
        'name': f"{name} ({feat_type})",
        'acc': accuracy_score(y_test, y_pred),
        'prec': precision_score(y_test, y_pred),
        'rec': recall_score(y_test, y_pred),
        'f1': f1_score(y_test, y_pred),
        'auc': roc_auc_score(y_test, y_prob),
        'y_prob': y_prob,
        'cm': confusion_matrix(y_test, y_pred),
        'coef': model.coef_[0] if hasattr(model, 'coef_') else None,
        'feats': what_features if feat_type == 'WHAT' else how_features
    }

results = []
results.append(run_model(LogisticRegression(max_iter=500), X_w_train_sc, X_w_test_sc,
y_train, y_test, "LR", "WHAT"))
results.append(run_model(LogisticRegression(max_iter=500), X_h_train_sc, X_h_test_sc,
y_train, y_test, "LR", "HOW"))
results.append(run_model(GaussianNB(), X_w_train, X_w_test, y_train, y_test, "NB", "WHAT"))
results.append(run_model(GaussianNB(), X_h_train, X_h_test, y_train, y_test, "NB", "HOW"))

log("Generating plots...")
```

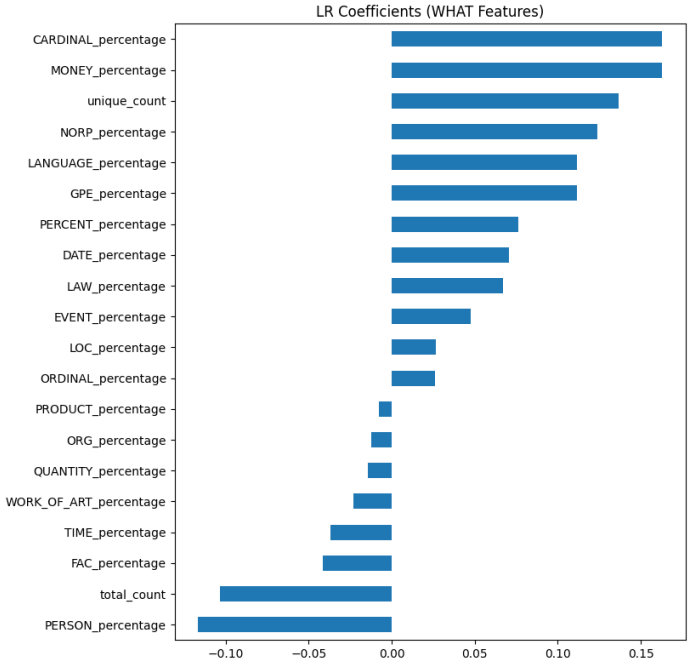
```
# 1. Plot ROC Curves
plt.figure(figsize=(10, 6))
for r in results:
    fpr, tpr, _ = roc_curve(y_test, r['y_prob'])
    plt.plot(fpr, tpr, label=f"{r['name']} (AUC = {r['auc']:.3f})")
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves: WHAT vs HOW')
plt.legend()
plt.savefig('roc_comparison.png')
plt.close()

# 2. Plot Confusion Matrices
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
for i, r in enumerate(results):
    sns.heatmap(r['cm'], annot=True, fmt='d', cmap='Blues', ax=axes[i//2, i%2])
    axes[i//2, i%2].set_title(r['name'])
plt.tight_layout()
plt.savefig('confusion_matrices.png')
plt.close()
```

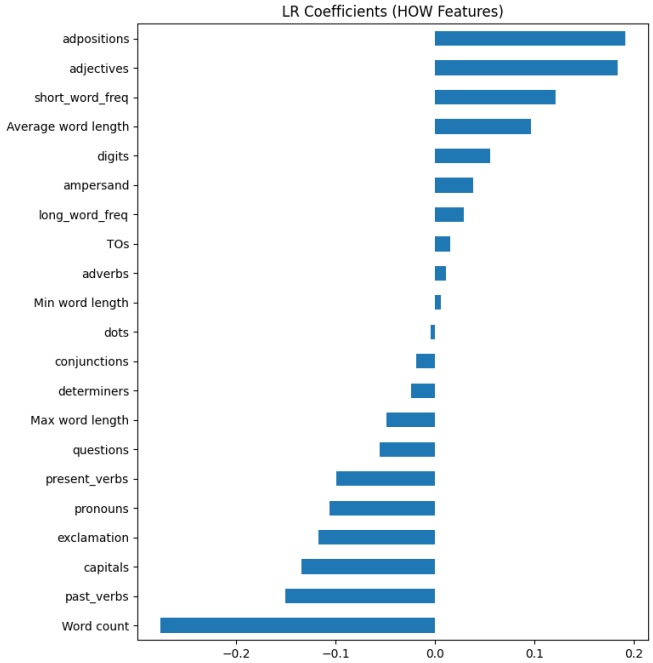
```
# 3. Feature Importance (LR)
lr_whats = results[0]
lr_hows = results[1]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 8))
pd.Series(lr_whats['coef'], index=lr_whats['feats']).sort_values().plot(kind='barh', ax=ax1)
ax1.set_title("LR Coefficients (WHAT Features)")
pd.Series(lr_hows['coef'], index=lr_hows['feats']).sort_values().plot(kind='barh', ax=ax2)
ax2.set_title("LR Coefficients (HOW Features)")
plt.tight_layout()
plt.savefig('feature_importance.png')
plt.close()

log("\nSUMMARY TABLE:")
res_df = pd.DataFrame(results).drop(columns=['y_prob', 'cm', 'coef', 'feats'])
print(res_df.to_string(index=False))
log("\nDone.")
```

Visualization




LR (WHAT)
9918
9663
7302
13377
LR (HOW)
9504
10077
6505
14174
NB (WHAT)
15889
3692
14298
6381
NB (HOW)
7063
12518
4955
15724


LR Coefficients (WHAT Features)
CARDINAL_percentage
MONEY_percentage
unique_count
NORP_percentage
LANGUAGE_percentage
GPE_percentage
PERCENT_percentage
DATE_percentage
LAW_percentage
EVENT_percentage
LOC_percentage
ORDINAL_percentage
PRODUCT_percentage
ORG_percentage
QUANTITY_percentage
WORK_OF_ART_percentage
TIME_percentage
FAC_percentage
total_count
PERSON_percentage


LR Coefficients (HOW Features)
adpositions
adjectives
short_word_freq
Average word length
digits
ampersand
long_word_freq
TOs
adverbs
Min word length
dots
conjunctions
determiners
Max word length
questions
present_verbs
pronouns
exclamation
capitals
past_verbs
Word count